



University
of Exeter

COMM113

Deep Learning – Week 4

Regularisations II

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Questions

More details in Week 11: Trustworthy DL

I would like to know more about label smoothing

I would like to know the flow how and where to use all these techniques to make a deep learning model.

The other regularisation methods

After we subtract the mean and apply relu we will see alot of zero. Is that good for the model.

Regularisers for Robustness

When and Where

Regularisation helps prevent **overfitting** by adding constraints or noise during training.

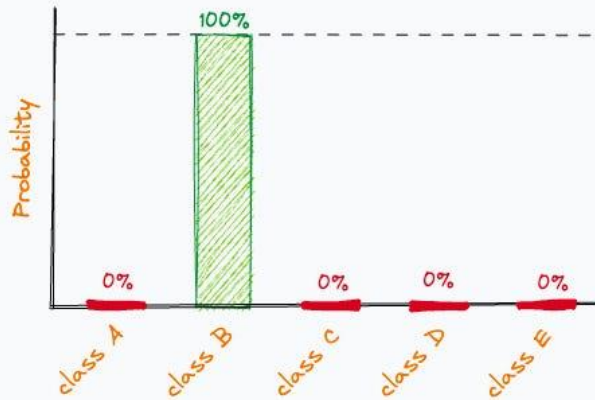
Different techniques address different sources of overfitting:

- **L1/L2** → control model weight magnitude
- **Dropout** → randomise neurons to prevent co-adaptation
- **Data Augmentation** → enrich dataset diversity
- **Label Smoothing** → prevent overconfident predictions
- **Early Stopping** → stop training before overfitting happens

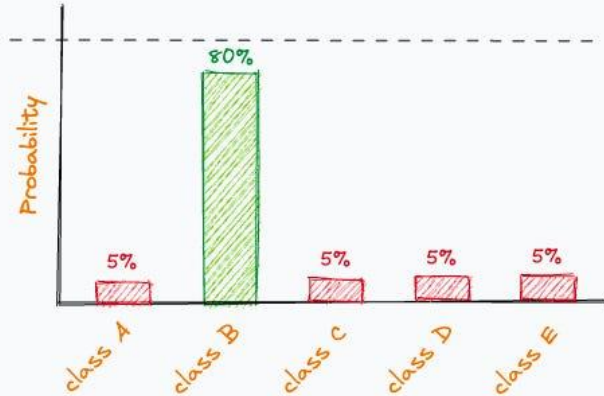
Label Smoothing

Label Smoothing

Instance #1



Instance #1



When and Where (Label Smoothing)

When to Use

- When labels are **noisy or ambiguous** (e.g., crowd-annotated datasets).
- To prevent **overconfident predictions** and improve calibration.
- Common in **classification with many classes** (e.g., ImageNet, language models).

Where to Use

- Replace one-hot target `[0, 0, 1, 0, 0]` with a softened version (e.g., `[0.05, 0.05, 0.8, 0.05, 0.05]`).
- Often used with **Cross-Entropy loss**.

Intuition

Reduces the “sharpness” of probability distributions, helping the model avoid overfitting to specific labels.

Also improves **out-of-distribution calibration**.

When and Where (L1 and L2)

When to Use

- **L1 (Lasso)**: when you suspect many weights should be **zero** (feature selection or sparsity desired).
- **L2 (Ridge / Weight Decay)**: when you want **small but non-zero** weights (smooth models, stable gradients).

Where to Use

- In **dense layers** or **linear models** (e.g., logistic regression, fully connected layers).
- In **CNNs**, often use **L2 (weight decay)** to control the norm of filters.
- In **transformers**, L2 weight decay is standard (e.g., AdamW with `weight_decay=0.01`).

Intuition

- L1 → drives some weights exactly to zero → **sparse representation**
- L2 → keeps weights small → **smooth decision boundary**

When and Where (Dropout)

When to Use

- When the network is **deep or has many neurons** (risk of co-adaptation).
- Works best in **fully connected layers** (less effective in convolutional or attention layers).

Where to Use

- Between hidden layers in feed-forward networks.
- Before classifier head in CNNs.
- Not used during inference (automatically disabled in `model.eval()`).

Intuition

Randomly drops neurons (with prob. p), forcing others to learn robust representations.

Acts like **ensemble averaging** of many subnetworks.

When and Where(Data Augmentation)

When to Use

- When training data is **limited or imbalanced**.
- To improve **invariance** (e.g., to rotation, scale, lighting).
- To simulate **real-world variability**.

Where to Use

- Especially effective for **images, audio, and text**:
 - Images → flip, rotate, crop, jitter, blur
 - Text → synonym replacement, back-translation
 - Audio → pitch shift, noise injection

Intuition

Model sees a new version of the same example every epoch → **generalises better**.

When and Where (Early Stopping)

When to Use

- When validation loss stops improving but training loss keeps decreasing → model starts overfitting.
- Works well when you have a reliable validation set.

Where to Use

- With any model that trains over epochs (NNs, gradient boosting, etc.).

Intuition

Avoids overfitting by stopping training **at the optimal generalisation point**.

When and Where (BN)

When to Use

- In deep networks where activations shift during training (“internal covariate shift”).
- When gradients are unstable or learning rate is difficult to tune.
- Particularly effective in CNNs, MLPs, and some transformer variants.

Where to Use

- Insert **between linear/convolution layers and non-linear activations** (e.g., after Conv, before ReLU).
- Usually applied to **hidden layers**, not input or output layers.

Intuition

For each mini-batch:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

BN normalises inputs, then scales and shifts them using learnable parameters (γ, β).

This keeps activation distributions stable → faster convergence and better generalisation.

When and Where (Summary)

Technique	Purpose	When to Use	Typical Placement / Scenario	Example Hyperparams / Notes
L1 Regularisation	Enforce sparsity, feature selection	Sparse models, small datasets	Linear or dense layers	$\lambda = 1\text{e-}5 - 1\text{e-}3$
L2 (Weight Decay)	Control weight magnitude, smooth boundary	Almost always used	All param layers (CNNs, Transformers)	$\lambda = 1\text{e-}5 - 1\text{e-}2$
Dropout	Prevent neuron co-adaptation	Deep/large networks	Between dense layers, before classifier head	$p = 0.3 - 0.5$
Data Augmentation	Increase data diversity	Small or imbalanced datasets	Input preprocessing stage	Random flip, crop, jitter, noise
Label Smoothing	Prevent overconfidence, improve calibration	Noisy / large-class datasets	Classification head with Cross-Entropy	$\epsilon = 0.05 - 0.2$
Early Stopping	Stop before overfitting	Any model with validation set	Training loop monitoring val loss	patience = 3–10 epochs
Batch Normalisation	Stabilise activations, speed up training	Deep CNNs / MLPs	Between linear/conv and ReLU	momentum = 0.1, $\epsilon = 1\text{e-}5$

Why BN *before* ReLU Works Best

After we subtract the mean and apply relu we will see a lot of zero. Is that good for the model.

(a) BN normalises activations to zero mean and unit variance

Before BN, activations might be very skewed or have exploding variance.

BN recentres and rescales them:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

so roughly **half the values are positive and half are negative** (zero mean).

Then ReLU cuts off the negative half → about **50% sparsity** (half zeros), which is *healthy and expected*.

Why BN *before* ReLU Works Best

After we subtract the mean and apply relu we will see a lot of zero. Is that good for the model.

(b) After ReLU, the distribution becomes highly skewed

If you put BN *after* ReLU:

(Affine $\rightarrow$ ReLU $\rightarrow$ BatchNorm)

then all negative values are zero, so the mean shifts positive, and variance drops.

BN will try to **re-centre this skewed distribution to zero**, reintroducing negative values (via its learned shift β), which **partially undoes** what ReLU just did.

That's redundant and can destabilise gradient flow.

In other words:

BN after ReLU “fights” the nonlinearity it just saw.

Why BN *before* ReLU Works Best

After we subtract the mean and apply relu we will see a lot of zero. Is that good for the model.

(c) Empirical observation

- **BN before ReLU** → stable training, faster convergence, better accuracy.
- **BN after ReLU** → sometimes works, but tends to yield slightly worse generalisation and slower learning.
- **BN after ReLU** is occasionally seen in older architectures (e.g. early VGG-BN variants), but most modern networks avoid it.

When Too Many Zeros Can Be a Problem

After we subtract the mean and apply relu we will see alot of zero. Is that good for the model.

- If ReLU output is almost all zeros (dead neurons):
- Often happens when learning rate is too high, weights poorly initialised, or BN is missing.
- You can mitigate by using BN before ReLU, smaller LR, or smoother activations (LeakyReLU, GELU, SiLU).

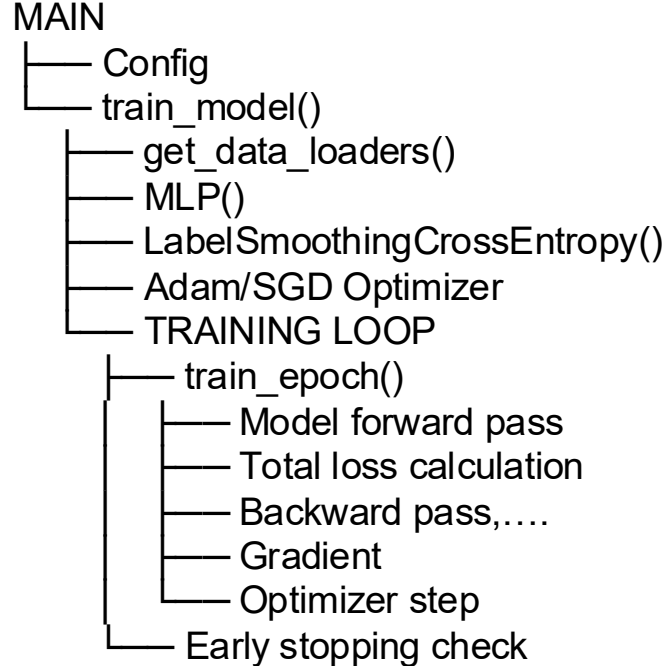
When Too Many Zeros Can Be a Problem

After we subtract the mean and apply relu we will see a lot of zero. Is that good for the model.

Design Choice	Effect	Recommendation
BN before ReLU	Balanced activations, stable gradients	✅ Default choice
BN after ReLU	Skewed activations, less stable	⚠️ Rarely better
Zeros after ReLU	Expected sparsity (~50%)	✅ Healthy
Too many zeros	Dead neurons, weak gradient flow	⚠️ Check learning rate or init

Flow Chart

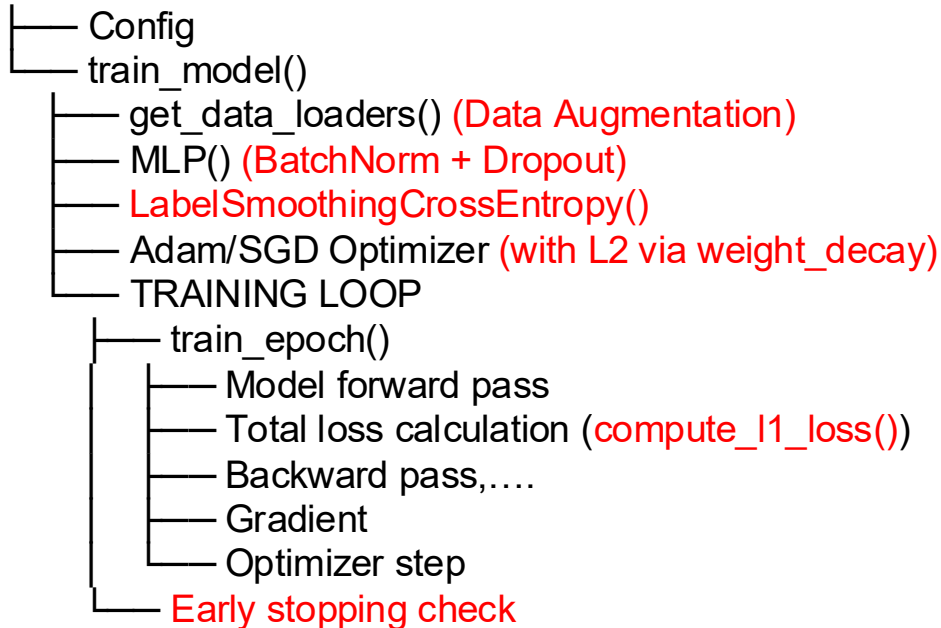
Implement regularisers in which function/class?



Flow Chart

Implement regularisers in which function/class?

MAIN



Config

```
# =====  
# CONFIGURATION  
# =====  
class Config:  
    # Training parameters  
    batch_size = 64  
    learning_rate = 0.001  
    num_epochs = 50  
  
    # Regularization parameters  
    l1_lambda = 0.0001      # L1 regularization strength  
    l2_lambda = 0.001      # L2 regularization (weight decay)  
    dropout_rate = 0.5      # Dropout probability  
    label_smooth_eps = 0.1 # Label smoothing epsilon  
  
    # Early stopping  
    patience = 10           # Early stopping patience  
    min_delta = 0.001      # Minimum improvement to be considered
```

Load data

Data augmentation

```
def get_data_loaders(config):  
    # Data augmentation for training set  
    train_transform = transforms.Compose([  
        transforms.RandomHorizontalFlip(p=0.5),  
        transforms.RandomRotation(degrees=10),  
        transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),  
        transforms.ToTensor(),  
        transforms.Normalize((0.5,), (0.5,)),  
        # Random erasing (another augmentation technique)  
        transforms.RandomErasing(p=0.2, scale=(0.02, 0.33))  
    ])   
  
    # No augmentation for test set  
    test_transform = transforms.Compose([  
        transforms.ToTensor(),  
        transforms.Normalize((0.5,), (0.5,))  
    ])
```

Why apply augmentation only on training data, not validation?

```
# Load datasets  
train_dataset = torchvision.datasets.FashionMNIST(  
    root='./data',  
    train=True,  
    download=True,  
    transform=train_transform  
)  
  
test_dataset = torchvision.datasets.FashionMNIST(  
    root='./data',  
    train=False,  
    download=True,  
    transform=test_transform  
)  
  
# Create data loaders  
train_loader = DataLoader(  
    train_dataset,  
    batch_size=config.batch_size,  
    shuffle=True,  
    num_workers=2  
)  
  
test_loader = DataLoader(  
    test_dataset,  
    batch_size=config.batch_size,  
    shuffle=False,  
    num_workers=2  
)  
  
return train_loader, test_loader
```

Load data

Data augmentation

```
def get_data_loaders(config):  
    # Data augmentation for training set  
    train_transform = transforms.Compose([  
        transforms.RandomHorizontalFlip(p=0.5),  
        transforms.RandomRotation(degrees=10),  
        transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),  
        transforms.ToTensor(),  
        transforms.Normalize((0.5,), (0.5,)),  
        # Random erasing (another augmentation technique)  
        transforms.RandomErasing(p=0.2, scale=(0.02, 0.33))  
    ])   
  
    # No augmentation for test set  
    test_transform = transforms.Compose([  
        transforms.ToTensor(),  
        transforms.Normalize((0.5,), (0.5,))  
    ])
```

Why apply augmentation only on training data, not validation?

Validation must reflect real-world performance

```
# Load datasets  
train_dataset = torchvision.datasets.FashionMNIST(  
    root='./data',  
    train=True,  
    download=True,  
    transform=train_transform  
)  
  
test_dataset = torchvision.datasets.FashionMNIST(  
    root='./data',  
    train=False,  
    download=True,  
    transform=test_transform  
)  
  
# Create data loaders  
train_loader = DataLoader(  
    train_dataset,  
    batch_size=config.batch_size,  
    shuffle=True,  
    num_workers=2  
)  
  
test_loader = DataLoader(  
    test_dataset,  
    batch_size=config.batch_size,  
    shuffle=False,  
    num_workers=2  
)  
  
return train_loader, test_loader
```

Network

How to package the defined layers in sequence?

```
class RegularizedMLP(nn.Module):  
    def __init__(self, dropout_rate=0.5):  
        super(RegularizedMLP, self).__init__()  
  
        self.flatten = nn.Flatten()  
  
        # Network layers with dropout  
        self.fc1 = nn.Linear(28 * 28, 512)  
        self.dropout1 = nn.Dropout(dropout_rate)  
  
        self.fc2 = nn.Linear(512, 256)  
        self.dropout2 = nn.Dropout(dropout_rate)  
  
        self.fc3 = nn.Linear(256, 128)  
        self.dropout3 = nn.Dropout(dropout_rate)  
  
        self.fc4 = nn.Linear(128, 10)  
  
        self.relu = nn.ReLU()  
  
        # Batch normalization (optional additional regularization)  
        self.bn1 = nn.BatchNorm1d(512)  
        self.bn2 = nn.BatchNorm1d(256)  
        self.bn3 = nn.BatchNorm1d(128)
```

Dropout and BN

Network

```
class RegularizedMLP(nn.Module):  
    def __init__(self, dropout_rate=0.5):  
        super(RegularizedMLP, self).__init__()  
  
        self.flatten = nn.Flatten()  
  
        # Network layers with dropout  
        self.fc1 = nn.Linear(28 * 28, 512)  
        self.dropout1 = nn.Dropout(dropout_rate)  
  
        self.fc2 = nn.Linear(512, 256)  
        self.dropout2 = nn.Dropout(dropout_rate)  
  
        self.fc3 = nn.Linear(256, 128)  
        self.dropout3 = nn.Dropout(dropout_rate)  
  
        self.fc4 = nn.Linear(128, 10)  
  
        self.relu = nn.ReLU()  
  
        # Batch normalization (optional additional regularization)  
        self.bn1 = nn.BatchNorm1d(512)  
        self.bn2 = nn.BatchNorm1d(256)  
        self.bn3 = nn.BatchNorm1d(128)
```

Dropout and BN

```
def forward(self, x):  
    x = self.flatten(x)  
  
    x = self.fc1(x)  
    x = self.bn1(x)  
    x = self.relu(x)  
    x = self.dropout1(x)  
  
    x = self.fc2(x)  
    x = self.bn2(x)  
    x = self.relu(x)  
    x = self.dropout2(x)  
  
    x = self.fc3(x)  
    x = self.bn3(x)  
    x = self.relu(x)  
    x = self.dropout3(x)  
  
    x = self.fc4(x)  
    return x
```

During evaluation, what happens to Dropout and BN?

Network

```
class RegularizedMLP(nn.Module):
    def __init__(self, dropout_rate=0.5):
        super(RegularizedMLP, self).__init__()

        self.flatten = nn.Flatten()

        # Network layers with dropout
        self.fc1 = nn.Linear(28 * 28, 512)
        self.dropout1 = nn.Dropout(dropout_rate)

        self.fc2 = nn.Linear(512, 256)
        self.dropout2 = nn.Dropout(dropout_rate)

        self.fc3 = nn.Linear(256, 128)
        self.dropout3 = nn.Dropout(dropout_rate)

        self.fc4 = nn.Linear(128, 10)

        self.relu = nn.ReLU()

        # Batch normalization (optional additional regularization)
        self.bn1 = nn.BatchNorm1d(512)
        self.bn2 = nn.BatchNorm1d(256)
        self.bn3 = nn.BatchNorm1d(128)
```

Dropout and BN

```
def forward(self, x):
    x = self.flatten(x)

    x = self.fc1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.dropout1(x)

    x = self.fc2(x)
    x = self.bn2(x)
    x = self.relu(x)
    x = self.dropout2(x)

    x = self.fc3(x)
    x = self.bn3(x)
    x = self.relu(x)
    x = self.dropout3(x)

    x = self.fc4(x)
    return x
```

During evaluation, what happens to Dropout and BN?

Dropout is disabled; BN uses running statistics.

Label Smoothing Loss

```
class LabelSmoothingCrossEntropy(nn.Module):
    def __init__(self, epsilon=0.1, reduction='mean'):
        super().__init__()
        self.epsilon = epsilon
        self.reduction = reduction

    def forward(self, preds, target):
        n_classes = preds.size(-1)
        log_preds = nn.functional.log_softmax(preds, dim=-1)

        # One-hot encode targets
        targets_one_hot = torch.zeros_like(log_preds).scatter_(1, target.unsqueeze(1), 1)

        # Apply label smoothing
        targets_smooth = targets_one_hot * (1 - self.epsilon) + self.epsilon / n_classes

        # Calculate loss
        loss = (-targets_smooth * log_preds).sum(dim=-1)

        if self.reduction == 'mean':
            loss = loss.mean()
        elif self.reduction == 'sum':
            loss = loss.sum()

        return loss
```

Soften the hard label

L1 Loss

```
def compute_l1_loss(model, l1_lambda):  
    l1_loss = 0  
    for param in model.parameters():  
        l1_loss += torch.abs(param).sum()  
    return l1_lambda * l1_loss
```

Early Stopping

```
class EarlyStopping:
    def __init__(self, patience=10, min_delta=0.001, mode='min'):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_score = None
        self.early_stop = False
        self.mode = mode

    def __call__(self, val_score):
        if self.mode == 'min':
            score = -val_score
        else:
            score = val_score

        if self.best_score is None:
            self.best_score = score
        elif score < self.best_score + self.min_delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_score = score
            self.counter = 0

        return self.early_stop
```

1. Initialise parameters:

Set patience (allowed epochs without improvement), minimum delta (required improvement), and mode ("min" for loss or "max" for accuracy).

2. Start training:

Track the best validation score seen so far.

3. After each epoch:

Compare the current validation score with the best one.

4. Check improvement:

- If the new score improves by at least `min_delta`, update `best_score` and reset the counter.
- If not, increase the counter by one.

5. Patience check:

If the counter exceeds the patience value, trigger early stopping.

6. Stop training:

When early stopping is triggered, terminate training to prevent overfitting and save computation time.

7. Otherwise:

Continue training until improvement stops again or the maximum number of epochs is reached.

Train epoch

Why call it 'epoch' rather than 'iteration'?

```
def train_epoch(model, loader, criterion, optimizer, config):
    model.train()
    running_loss = 0.0
    running_l1_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(config.device), labels.to(config.device)

        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate losses
        ce_loss = criterion(outputs, labels)
        l1_loss = compute_l1_loss(model, config.l1_lambda)

        # Total loss = CrossEntropy + L1 regularization
        # Note: L2 is handled by weight_decay in optimizer
        total_loss = ce_loss + l1_loss

        # Backward pass and optimization
        total_loss.backward()
```

```
optimizer.step()
```

```
# Track metrics
```

```
running_loss += ce_loss.item()
```

```
running_l1_loss += l1_loss.item()
```

```
_, predicted = torch.max(outputs.data, 1)
```

```
total += labels.size(0)
```

```
correct += (predicted == labels).sum().item()
```

```
epoch_loss = running_loss / len(loader)
```

```
epoch_l1_loss = running_l1_loss / len(loader)
```

```
epoch_acc = 100 * correct / total
```

```
return epoch_loss, epoch_l1_loss, epoch_acc
```

Train epoch

```
def train_epoch(model, loader, criterion, optimizer, config):
    model.train()
    running_loss = 0.0
    running_l1_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(config.device), labels.to(config.device)

        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate losses
        ce_loss = criterion(outputs, labels)
        l1_loss = compute_l1_loss(model, config.l1_lambda)

        # Total loss = CrossEntropy + L1 regularization
        # Note: L2 is handled by weight_decay in optimizer
        total_loss = ce_loss + l1_loss

        # Backward pass and optimization
        total_loss.backward()
```

Why call it 'epoch' rather than 'iteration'?

An epoch covers the whole dataset once; an iteration covers one batch.

```
optimizer.step()
```

```
# Track metrics
```

```
running_loss += ce_loss.item()
```

```
running_l1_loss += l1_loss.item()
```

```
_, predicted = torch.max(outputs.data, 1)
```

```
total += labels.size(0)
```

```
correct += (predicted == labels).sum().item()
```

```
epoch_loss = running_loss / len(loader)
```

```
epoch_l1_loss = running_l1_loss / len(loader)
```

```
epoch_acc = 100 * correct / total
```

```
return epoch_loss, epoch_l1_loss, epoch_acc
```

Train epoch

```
def train_epoch(model, loader, criterion, optimizer, config):
    model.train()
    running_loss = 0.0
    running_l1_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(config.device), labels.to(config.device)

        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate losses
        ce_loss = criterion(outputs, labels)
        l1_loss = compute_l1_loss(model, config.l1_lambda)

        # Total loss = CrossEntropy + L1 regularization
        # Note: L2 is handled by weight_decay in optimizer
        total_loss = ce_loss + l1_loss

        # Backward pass and optimization
        total_loss.backward()
```

```
optimizer.step()
```

```
# Track metrics
```

```
running_loss += ce_loss.item()
```

```
running_l1_loss += l1_loss.item()
```

```
_, predicted = torch.max(outputs.data, 1)
```

```
total += labels.size(0)
```

```
correct += (predicted == labels).sum().item()
```

```
epoch_loss = running_loss / len(loader)
```

```
epoch_l1_loss = running_l1_loss / len(loader)
```

```
epoch_acc = 100 * correct / total
```

```
return epoch_loss, epoch_l1_loss, epoch_acc
```

Train epoch

```
def train_epoch(model, loader, criterion, optimizer, config):
```

```
    model.train()
```

```
    running_loss = 0.0
```

```
    running_l1_loss = 0.0
```

```
    correct = 0
```

```
    total = 0
```

```
    for images, labels in loader:
```

```
        images, labels = images.to(config.device), labels.to(config.device)
```

```
        optimizer.zero_grad()
```

```
        # Forward pass
```

```
        outputs = model(images)
```

```
        # Calculate losses
```

```
        ce_loss = criterion(outputs, labels)
```

```
        l1_loss = compute_l1_loss(model, config.l1_lambda)
```

```
        # Total loss = CrossEntropy + L1 regularization
```

```
        # Note: L2 is handled by weight_decay in optimizer
```

```
        total_loss = ce_loss + l1_loss
```

```
        # Backward pass and optimization
```

```
        total_loss.backward()
```

```
        optimizer.step()
```

```
        # Track metrics
```

```
        running_loss += ce_loss.item()
```

```
        running_l1_loss += l1_loss.item()
```

```
        _, predicted = torch.max(outputs.data, 1)
```

```
        total += labels.size(0)
```

```
        correct += (predicted == labels).sum().item()
```

```
    epoch_loss = running_loss / len(loader)
```

```
    epoch_l1_loss = running_l1_loss / len(loader)
```

```
    epoch_acc = 100 * correct / total
```

```
    return epoch_loss, epoch_l1_loss, epoch_acc
```

What does it mean?

Train epoch

```
def train_epoch(model, loader, criterion, optimizer, config):
    model.train()
    running_loss = 0.0
    running_l1_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(config.device), labels.to(config.device)

        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate losses
        ce_loss = criterion(outputs, labels)
        l1_loss = compute_l1_loss(model, config.l1_lambda)

        # Total loss = CrossEntropy + L1 regularization
        # Note: L2 is handled by weight_decay in optimizer
        total_loss = ce_loss + l1_loss

        # Backward pass and optimization
        total_loss.backward()
```

`optimizer.step()`

What does it mean?

```
# Track metrics
running_loss += ce_loss.item()
running_l1_loss += l1_loss.item()
_, predicted = torch.max(outputs.data, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(loader)
epoch_l1_loss = running_l1_loss / len(loader)
epoch_acc = 100 * correct / total

return epoch_loss, epoch_l1_loss, epoch_acc
```

Train Model

Full Pipeline: we tie everything together

```
def train_model(config):  
    # Initialize everything  
    train_loader, test_loader = get_data_loaders(config)  
  
    model = RegularizedMLP(dropout_rate=config.dropout_rate).to(config.device)  
  
    # Loss function with label smoothing  
    criterion = LabelSmoothingCrossEntropy(epsilon=config.label_smooth_eps)  
  
    # Optimizer with L2 regularization (weight decay)  
    optimizer = optim.Adam(  
        model.parameters(),  
        lr=config.learning_rate,  
        weight_decay=config.l2_lambda # L2 regularization  
    )  
  
    # Learning rate scheduler (additional regularization technique)  
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
        optimizer, mode='min', factor=0.5, patience=5, verbose=True  
    )  
  
    # Early stopping  
    early_stopping = EarlyStopping(patience=config.patience, min_delta=config.min_de
```

Call the model

Train Model

```
def train_model(config):  
    # Initialize everything  
    train_loader, test_loader = get_data_loaders(config)  
  
    model = RegularizedMLP(dropout_rate=config.dropout_rate).to(config.device)  
  
    # Loss function with label smoothing  
    criterion = LabelSmoothingCrossEntropy(epsilon=config.label_smooth_eps)  
  
    # Optimizer with L2 regularization (weight decay)  
    optimizer = optim.Adam(  
        model.parameters(),  
        lr=config.learning_rate,  
        weight_decay=config.l2_lambda # L2 regularization  
    )  
  
    # Learning rate scheduler (additional regularization technique)  
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
        optimizer, mode='min', factor=0.5, patience=5, verbose=True  
    )  
  
    # Early stopping  
    early_stopping = EarlyStopping(patience=config.patience, min_delta=config.min_de
```

Call the label smoothing cross-entropy loss

Train Model

```
def train_model(config):  
    # Initialize everything  
    train_loader, test_loader = get_data_loaders(config)  
  
    model = RegularizedMLP(dropout_rate=config.dropout_rate).to(config.device)  
  
    # Loss function with label smoothing  
    criterion = LabelSmoothingCrossEntropy(epsilon=config.label_smooth_eps)  
  
    # Optimizer with L2 regularization (weight decay)  
    optimizer = optim.Adam(  
        model.parameters(),  
        lr=config.learning_rate,  
        weight_decay=config.l2_lambda # L2 regularization  
    )  
  
    # Learning rate scheduler (additional regularization technique)  
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
        optimizer, mode='min', factor=0.5, patience=5, verbose=True  
    )  
  
    # Early stopping  
    early_stopping = EarlyStopping(patience=config.patience, min_delta=config.min_de
```

How to implement L2 regulariser?

Train Model

```
def train_model(config):  
    # Initialize everything  
    train_loader, test_loader = get_data_loaders(config)  
  
    model = RegularizedMLP(dropout_rate=config.dropout_rate).to(config.device)  
  
    # Loss function with label smoothing  
    criterion = LabelSmoothingCrossEntropy(epsilon=config.label_smooth_eps)  
  
    # Optimizer with L2 regularization (weight decay)  
    optimizer = optim.Adam(  
        model.parameters(),  
        lr=config.learning_rate,  
        weight_decay=config.l2_lambda # L2 regularization  
    )  
  
    # Learning rate scheduler (additional regularization technique)  
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
        optimizer, mode='min', factor=0.5, patience=5, verbose=True  
    )  
  
    # Early stopping  
    early_stopping = EarlyStopping(patience=config.patience, min_delta=config.min_de
```

Training Model

```
for epoch in range(config.num_epochs):
```

```
    # Train
```

```
    train_loss, train_l1_loss, train_acc = train_epoch(
        model, train_loader, criterion, optimizer, config
    )
```

```
    # Evaluate
```

```
    test_loss, test_acc = evaluate(model, test_loader, criterion, config)
```

```
    # Store history
```

```
    history['train_loss'].append(train_loss)
    history['train_l1_loss'].append(train_l1_loss)
    history['train_acc'].append(train_acc)
    history['test_loss'].append(test_loss)
    history['test_acc'].append(test_acc)
```

```
    # Learning rate scheduling
```

```
    scheduler.step(test_loss)
```

```
    # Save best model
```

```
    if test_acc > best_test_acc:
        best_test_acc = test_acc
        best_model_state = model.state_dict().copy()
```

In training loop, calling
`train_epoch` once means
completing model update
over whole set once

```
    # Save best model
```

```
    if test_acc > best_test_acc:
        best_test_acc = test_acc
        best_model_state = model.state_dict().copy()
```

```
    # Print progress
```

```
    print(f"Epoch [{epoch+1}/{config.num_epochs}] | "
          f"Train Loss: {train_loss:.4f} (L1: {train_l1_loss:.4f}) | "
          f"Train Acc: {train_acc:.2f}% | "
          f"Test Loss: {test_loss:.4f} | "
          f"Test Acc: {test_acc:.2f}%")
```

```
    # Early stopping check
```

```
    if early_stopping(test_loss):
        print(f"\nEarly stopping triggered at epoch {epoch+1}")
        break
```

```
    # Load best model
```

```
    if best_model_state is not None:
        model.load_state_dict(best_model_state)
        print(f"\nBest Test Accuracy: {best_test_acc:.2f}%")
```

```
    return model, history
```

Training Model

```
for epoch in range(config.num_epochs):
    # Train
    train_loss, train_l1_loss, train_acc = train_epoch(
        model, train_loader, criterion, optimizer, config
    )

    # Evaluate
    test_loss, test_acc = evaluate(model, test_loader, criterion, config)

    # Store history
    history['train_loss'].append(train_loss)
    history['train_l1_loss'].append(train_l1_loss)
    history['train_acc'].append(train_acc)
    history['test_loss'].append(test_loss)
    history['test_acc'].append(test_acc)

    # Learning rate scheduling
    scheduler.step(test_loss)

    # Save best model
    if test_acc > best_test_acc:
        best_test_acc = test_acc
        best_model_state = model.state_dict().copy()
```

Check early stopping

```
# Save best model
if test_acc > best_test_acc:
    best_test_acc = test_acc
    best_model_state = model.state_dict().copy()

# Print progress
print(f"Epoch [{epoch+1}/{config.num_epochs}] | "
      f"Train Loss: {train_loss:.4f} (L1: {train_l1_loss:.4f}) | "
      f"Train Acc: {train_acc:.2f}% | "
      f"Test Loss: {test_loss:.4f} | "
      f"Test Acc: {test_acc:.2f}%")

# Early stopping check
if early_stopping(test_loss):
    print(f"\nEarly stopping triggered at epoch {epoch+1}")
    break

# Load best model
if best_model_state is not None:
    model.load_state_dict(best_model_state)
    print(f"\nBest Test Accuracy: {best_test_acc:.2f}%")

return model, history
```

Training Model

```
for epoch in range(config.num_epochs):
    # Train
    train_loss, train_l1_loss, train_acc = train_epoch(
        model, train_loader, criterion, optimizer, config
    )

    # Evaluate
    test_loss, test_acc = evaluate(model, test_loader, criterion, config)

    # Store history
    history['train_loss'].append(train_loss)
    history['train_l1_loss'].append(train_l1_loss)
    history['train_acc'].append(train_acc)
    history['test_loss'].append(test_loss)
    history['test_acc'].append(test_acc)

    # Learning rate scheduling
    scheduler.step(test_loss)

    # Save best model
    if test_acc > best_test_acc:
        best_test_acc = test_acc
        best_model_state = model.state_dict().copy()
```

Save best model

```
# Save best model
if test_acc > best_test_acc:
    best_test_acc = test_acc
    best_model_state = model.state_dict().copy()

# Print progress
print(f"Epoch [{epoch+1}/{config.num_epochs}] | "
      f"Train Loss: {train_loss:.4f} (L1: {train_l1_loss:.4f}) | "
      f"Train Acc: {train_acc:.2f}% | "
      f"Test Loss: {test_loss:.4f} | "
      f"Test Acc: {test_acc:.2f}%")

# Early stopping check
if early_stopping(test_loss):
    print(f"\nEarly stopping triggered at epoch {epoch+1}")
    break

# Load best model
if best_model_state is not None:
    model.load_state_dict(best_model_state)
    print(f"\nBest Test Accuracy: {best_test_acc:.2f}%")

return model, history
```

Q&A